PART 4 — TEAM ROLE ASSIGNMENTS

ROLE ASSIGNMENT PHILOSOPHY

Principle 1: Every member owns a module that the system CANNOT work without. **Principle 2:** At least 3 people (Ayush, Shaurya, Aditi) must understand the end-to-end system. **Principle 3:** Nobody is "just the PPT person" or "just the frontend person." **Principle 4:** Difficulty is calibrated to experience — challenging but achievable.

AYUSH — System Integrator + Agent Interface Lead

Attribute	Detail
Primary Role	System Integration + Agent Loop Architect
Secondary Role	Demo UI + End-to-End Pipeline Owner
Subsystem Ownership	Agent Planner (Module L), Action Executor (Module M), Demo UI, Integration glue
Why this role	Your frontend strength handles the demo UI, but your PRIMARY job is the agent loop — the Python orchestrator that ties perception, redaction, and execution together. This forces you to deeply understand every module.

Concepts to Master

Concept	Depth	Why
Agent loop (observe→think→act→verify)	Advanced	You BUILD this. It IS your module.
Playwright browser automation	Advanced	You control the browser
VLM prompt engineering	Intermediate	You write the prompts that drive perception + action

Concept	Depth	Why
Grounding (coordinates → clicks)	Intermediate	You translate VLM output to browser actions
Routing logic (local vs remote)	Intermediate	You implement the decision function
Privacy pipeline flow	Intermediate	You must route data through Abhishek's redaction engine
FastAPI + WebSocket	Intermediate	You serve the demo UI and stream agent status

Technologies to Learn

- Python async/await patterns
- Playwright Python API (advanced: CDP, accessibility tree)
- FastAPI (REST + WebSocket endpoints)
- OpenAI-compatible API client (for calling llama-server and remote models)
- JSON action parsing and validation

Implementation Responsibility

1. **Agent loop core** — The `while not done` loop that orchestrates everything
2. **Action parser** — Convert VLM text output → structured {action, target, params}
3. **Action executor** — Map structured actions → Playwright calls
4. **Routing logic** — Local vs remote decision function
5. **Demo UI** — Real-time dashboard showing agent workflow
6. **Integration** — Connect everyone's modules into one pipeline

Deliverables

- [ ] Working agent loop that takes a task → executes browser actions → reports completion
- [ ] Demo UI with original/detected/sanitized screenshot panels
- [ ] Real-time WebSocket feed of agent steps to UI
- [ ] Integration of Aditi's VLM perception + Abhishek's redaction into the loop

Dependencies

- Aditi's VLM perception module (screenshots → understanding)
- Abhishek's redaction engine (screenshots → sanitized screenshots)

- Shaurya's llama-server setup (VLM inference endpoint)
- Himanshu's API design (remote model communication)

Backup Person: Shaurya (understands the backend and can step into orchestration)

Difficulty: 7/10

Must Explain to Judge

- How the agent loop works end-to-end
- How routing decisions are made
- How actions are parsed and verified
- Why the architecture is modular
- The complete data flow from user task → browser action

SHAURYA — Backend + Model Infrastructure Lead

Attribute	Detail
Primary Role	Model Serving + Inference Infrastructure
Secondary Role	Performance Optimization + Architecture Co-Owner
Subsystem Ownership	VLM Server (Module D), Remote Model Integration (Module K), Verification (Module N), Performance
Why this role	Your powerful hardware + backend strength = you own the hardest infrastructure. You make the VLM run fast on-device.

Concepts to Master

Concept	Depth	Why
VLM inference (llama.cpp)	Advanced	You set up and optimize the local model server
GGUF quantization	Advanced	You choose and benchmark quantization levels

Concept	Depth	Why
VRAM management	Advanced	You ensure everything fits in 8 GB
GPU profiling	Intermediate	You measure and optimize inference latency
OpenAI API format	Intermediate	llama-server exposes this; you configure and test it
Remote model API integration	Intermediate	You handle the cloud model calls
Action verification	Intermediate	You implement before/after screenshot comparison

Technologies to Learn

- llama.cpp (building from source, server flags, GPU layer offloading)
- GGUF model format (downloading, converting, testing quantizations)
- nvidia-smi / GPU monitoring tools
- Python httpx (async HTTP client for remote API calls)
- SSIM / image comparison (for verification module)

Implementation Responsibility

1. **llama-server deployment** — Download model, configure, optimize, benchmark
2. **Quantization testing** — Q4_K_M vs Q5_K_M vs Q8, measure quality/speed tradeoff
3. **VRAM budget enforcement** — Ensure VLM + face detection + OCR fit in 8 GB
4. **Remote model client** — Async API calls to cloud model with sanitized data
5. **Verification module** — Screenshot diff to confirm actions succeeded
6. **Performance benchmarks** — Latency per component, VRAM usage tracking

Deliverables

- [ ] llama-server running Qwen3.5-4B with verified vision capability
- [ ] Benchmark report: latency × quantization × resolution
- [ ] Remote model API client with sanitized data handling
- [ ] Verification module (before/after screenshot comparison)
- [ ] VRAM usage monitor/dashboard

Dependencies

- Aditi's model selection (which exact model to serve)

- Abhishek's redaction output (sanitized screenshots for remote API)
- Ayush's agent loop (consuming inference results)

Backup Person: Ayush (understands the agent loop and can debug inference issues)

Difficulty: 8/10

Must Explain to Judge

- How quantization works and why Q4_K_M was chosen
- Exact VRAM breakdown
- Inference latency numbers with evidence
- How the local server exposes an OpenAI-compatible API
- How verification catches failed actions

HIMANSHU — Networking + API + Architecture Defense Lead

Attribute	Detail
Primary Role	API Design + Local↔Remote Communication
Secondary Role	Technical Documentation + Architecture Presentation
Subsystem Ownership	Routing Layer (Module J), Logging/Telemetry (Module O), API specification, Documentation
Why this role	Your networking knowledge maps directly to the most critical communication question: what data leaves the device, when, and how. You also own the architecture story for the jury.

Concepts to Master

Concept	Depth	Why
API design (REST, WebSocket)	Advanced	You design all internal and external APIs

Concept	Depth	Why
Data flow security	Advanced	You must trace every byte that leaves the device
Network protocols (HTTPS, WSS)	Intermediate	You ensure secure communication
Routing logic principles	Intermediate	You document and defend the local/remote decision
Privacy data classification	Intermediate	You categorize what is safe vs unsafe to transmit
Latency analysis	Intermediate	You measure network overhead
System architecture documentation	Advanced	You create the architecture diagrams for jury

Technologies to Learn

- FastAPI (API design, middleware, CORS)
- WebSocket protocol (real-time updates)
- httpx / aiohttp (async HTTP clients)
- Structured logging (Python logging + JSON formatters)
- Mermaid / draw.io (architecture diagrams)
- Wireshark basics (to demonstrate no raw PII leaves device — demo proof)

Implementation Responsibility

1. **API specification** — Define all endpoints, request/response schemas
2. **Privacy gateway** — Middleware that validates no raw PII in outbound requests
3. **Logging system** — Structured JSON logs with PII-safe filters
4. **Telemetry collector** — Timing data from all modules
5. **Architecture documentation** — Complete system diagrams for PPT/jury
6. **Network security demo** — Show (via logs or Wireshark) that only sanitized data transmits

Deliverables

- [ ] API specification document (OpenAPI/Swagger)
- [ ] Privacy gateway middleware (validates outbound sanitization)

- [ ] Structured logging system with PII filtering
- [ ] Telemetry dashboard (latency breakdown per module)
- [ ] Architecture diagrams (system, data flow, sequence, deployment)
- [ ] Technical documentation for the PPT

Dependencies

- Ayush's agent loop (integrates logging/telemetry)
- Shaurya's remote API client (applies privacy gateway)
- Abhishek's redaction output (validates before transmission)

Backup Person: Aditi (understands the theoretical architecture deeply)

Difficulty: 6/10

Must Explain to Judge

- Complete data flow: what moves where, when, and why
- Privacy guarantee: how we prove no raw PII leaves the device
- API design decisions (REST vs WebSocket, why)
- Latency breakdown by network vs compute
- Security model of local↔remote communication

ADITI — ML/VLM Research + Grounding Lead

Attribute	Detail
Primary Role	VLM Research + Visual Perception + Grounding
Secondary Role	Model Evaluation + Accuracy Optimization
Subsystem Ownership	Local VLM Perception (Module D), UI Grounding (Module E), OCR Integration (Module F), Model Evaluation
Why this role	Your academic strength + ability to learn deeply = you own the hardest RESEARCH problem. You decide which model we use and how to prompt it for maximum grounding accuracy.

Concepts to Master

Concept	Depth	Why
Vision-Language Models (architecture)	Advanced	You must explain HOW the model sees and understands images
Visual grounding	Advanced	Core capability — mapping language → coordinates
Prompt engineering for VLMs	Advanced	Different prompts dramatically change grounding accuracy
Quantization effects on accuracy	Intermediate	You verify the model stays accurate after quantization
OCR integration	Intermediate	You connect OCR output to PII detection
UI element taxonomy	Intermediate	Buttons, inputs, dropdowns, modals — what the model must recognize
Evaluation metrics (IoU, accuracy)	Advanced	You define and measure model quality

Technologies to Learn

- Hugging Face Hub (model search, downloading, model cards)
- llama.cpp integration (sending vision requests to llama-server)
- OpenCV / Pillow (image preprocessing for VLM input)
- Evaluation scripting (IoU calculation, accuracy metrics)
- Jupyter notebooks (for rapid experimentation)

Implementation Responsibility

1. **Model selection** — Evaluate Qwen3.5-4B vs alternatives, final recommendation
2. **Prompt design** — Craft prompts for: page description, element grounding, action suggestion
3. **Grounding module** — Take screenshot + "click the login button" → return coordinates
4. **OCR integration** — VLM-based OCR or PaddleOCR setup
5. **Accuracy evaluation** — Benchmark grounding accuracy on test screenshots
6. **Image preprocessing** — Resolution scaling, format optimization for VLM input

Deliverables

- [ ] Model comparison report (Qwen3.5-4B vs SmolVLM vs MiniCPM-V)
- [ ] Prompt library for perception, grounding, and action generation
- [ ] Grounding module: input(screenshot, "element description") → output(x, y, w, h)
- [ ] Grounding accuracy benchmarks (IoU scores on test set)
- [ ] OCR integration for text extraction from screenshots

Dependencies

- Shaurya's llama-server (model must be running to test)
- Ayush's agent loop (consumes grounding output)
- Abhishek's PII detection (consumes OCR output)

Backup Person: Shaurya (understands model serving, can debug inference issues)

Difficulty: 9/10 (hardest research role)

Must Explain to Judge

- How VLMs process images (vision encoder → projection → language model)
- What grounding is and how we achieve it
- Why Qwen3.5-4B was chosen (with benchmark data)
- Quantization tradeoffs (accuracy vs speed)
- What happens when grounding fails (fallback to DOM)
- Difference between our approach and traditional CV

ABHISHEK — Privacy + Redaction + Testing Lead

Attribute	Detail
Primary Role	Redaction Engine + Privacy Validation
Secondary Role	Benchmarking + Testing Framework
Subsystem Ownership	PII Detection (Module G), Face Detection (Module H), Redaction (Module I), Benchmarking (Module Q)
Why this role	Your discipline + work ethic maps perfectly to the most MEASURABLE module. Every output has a clear success/failure metric.

Concepts to Master

Concept	Depth	Why
PII patterns (Aadhaar, PAN, email, phone)	Advanced	You write every detection rule
Regex patterns (+ Verhoeff checksum)	Advanced	Core detection technology
Face detection (MediaPipe)	Intermediate	You integrate and test BlazeFace
Image redaction techniques	Intermediate	You implement blur, mask, pixelation
DOM attribute scanning	Intermediate	You extract sensitive field types from DOM
Precision/recall metrics	Advanced	You measure and report detection quality
Benchmarking methodology	Advanced	You design and run all benchmarks

Technologies to Learn

- Python `re` module (regex, advanced patterns)
- MediaPipe Face Detection Python API
- OpenCV (rectangle drawing, Gaussian blur, image manipulation)
- Pillow (image masking, composite operations)
- Verhoeff algorithm (Aadhaar validation)
- Luhn algorithm (credit card validation)
- pytest (test framework)
- psutil + nvidia-smi (resource monitoring)

Implementation Responsibility

1. **DOM PII scanner** — Parse DOM tree, flag sensitive fields by attribute
2. **Regex PII engine** — Detect Aadhaar, PAN, email, phone, credit card in text
3. **Face detection module** — MediaPipe integration, face bounding boxes
4. **Redaction engine** — Apply masks/blur to all detected sensitive regions
5. **Redaction verifier** — Re-scan sanitized image to confirm no PII remains
6. **Benchmark framework** — Latency profiling, accuracy measurement, resource monitoring

7. **Test suite** — Unit tests for every PII pattern, edge cases

Deliverables

- [ ] PII detection engine: DOM scanning + regex + face detection
- [ ] Redaction engine: input(screenshot, PII regions) → output(sanitized screenshot)
- [ ] Verification module: re-scan sanitized screenshot for missed PII
- [ ] Benchmark results: precision, recall, F1, latency per PII type
- [ ] Test suite: 50+ test cases covering all PII types + edge cases
- [ ] Resource usage report: VRAM, RAM, CPU per module

Dependencies

- Aditi's OCR output (text to scan for PII patterns)
- Ayush's agent loop (calling redaction in the pipeline)
- Shaurya's server (GPU availability for face detection)

Backup Person: Himanshu (understands the privacy/security layer)

Difficulty: 7/10

Must Explain to Judge

- All PII detection techniques and their tradeoffs
- What Verhoeff checksum is and why it matters for Aadhaar
- Precision vs recall in redaction (why recall is more important)
- How we handle PII that appears as rendered text (not in DOM)
- Benchmark results with specific numbers
- What happens when detection fails (false negatives)

NIDHI — QA + Documentation + Demo Lead

Attribute	Detail
Primary Role	Quality Assurance + Test Scenario Design
Secondary Role	Documentation + Demo Script + Presentation
Subsystem Ownership	Test Scenarios, Demo Scripts, Documentation, Result Collection

Attribute	Detail
Why this role	Your communication strength + organization skills = you own the FACE of the project. Your work directly determines demo quality and jury impression.

Concepts to Master

Concept	Depth	Why
System architecture (high-level)	Basic-Intermediate	You must explain the overall system
Browser agent concept	Basic	Understand what the agent does
Privacy/redaction flow	Basic-Intermediate	Understand what gets redacted and why
Test case design	Intermediate	You design test scenarios
Benchmark result interpretation	Basic	You collect and present numbers
SIH presentation format	Advanced	You optimize the jury presentation

Technologies to Learn

- Basic Python (reading/running test scripts)
- HTML basics (to create test web pages with PII)
- Markdown (documentation)
- Screenshot/recording tools (for demo video)
- Git basics (clone, pull, push, basic workflow)

Implementation Responsibility

1. **Test web pages** — Create 5–10 HTML pages with various PII scenarios (password fields, emails, faces, Aadhaar numbers, etc.)
2. **Test scenario matrix** — Document every test case: input, expected detection, expected redaction
3. **Demo script** — Write the exact 3–5 minute demo narrative with timing
4. **Result collection** — Gather benchmark numbers from Abhishek, format into tables/charts

5. **Documentation** — README, setup guide, architecture overview (with Himanshu)
6. **Issue tracking** — Maintain GitHub Issues board, track task completion
7. **Presentation support** — Help Himanshu create jury-ready slides

Deliverables

- [ ] 5–10 test HTML pages with diverse PII scenarios
- [ ] Test scenario matrix (spreadsheet/markdown table)
- [ ] Written demo script with timing marks
- [ ] Formatted benchmark results (tables + charts)
- [ ] Project README.md
- [ ] GitHub Issues board populated with all tasks
- [ ] PPT support content (screenshots, flow diagrams, results)

Dependencies

- Abhishek's benchmark data (formats results from his framework)
- All team members (collects deliverables for documentation)
- Himanshu (collaborates on architecture docs and PPT)

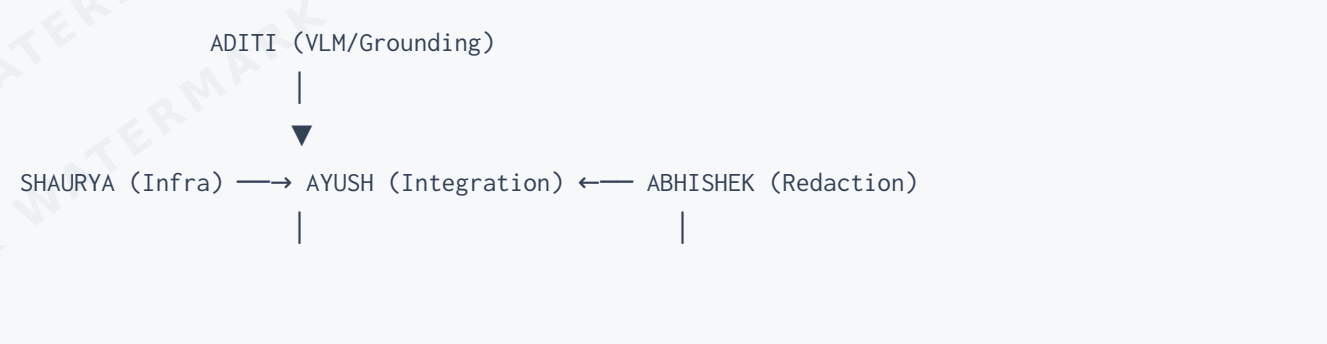
Backup Person: Himanshu (documentation and presentation overlap)

Difficulty: 4/10 (lower technical bar, but high IMPACT on demo quality)

Must Explain to Judge

- What the system does at a high level
- The demo walkthrough (what the audience is seeing)
- What test scenarios were tested
- High-level privacy guarantees (in non-technical language)
- How we validated our claims (point to benchmarks)

ROLE DEPENDENCY GRAPH



Critical path: Shaurya's llama-server → Aditi's grounding → Ayush's agent loop → Abhishek's redaction → Working MVP

CROSS-TRAINING MATRIX (WHO UNDERSTANDS WHAT)

Module	Ayush	Shaurya	Himanshu	Aditi	Abhishek	Nidhi
Agent Loop	**OWNS**	Deep	Basic	Basic	Basic	Basic
VLM Inference	Deep	**OWNS**	Basic	Deep	Basic	Aware
Grounding	Deep	Intermediate	Basic	**OWNS**	Basic	Aware
Redaction	Intermediate	Basic	Basic	Basic	**OWNS**	Aware
API/Networking	Intermediate	Intermediate	**OWNS**	Basic	Basic	Aware
Demo/Docs	Intermediate	Basic	Deep	Basic	Basic	**OWNS**